

Introduction à MongoDB

Table des matières

I.	Introduction.....	3
A.	Qu'est-ce qu'une base de données ?.....	3
B.	Bases de données relationnelles ou non relationnelles.....	3
1.	Base de données relationnelles	3
2.	Base de données non relationnelles	3
3.	Comparatif	3
4.	Types d'évolutivité.....	3
II.	Grands concepts	4
A.	Le théorème CAP	4
1.	Consistency (Cohérence des données)	4
2.	Availability (Disponibilité)	4
3.	Partition Tolerance (Tolérance au partitionnement).....	4
4.	Les compromis	5
B.	Le modèle ACID	5
1.	Atomicité	5
2.	Cohérence	5
3.	Isolation.....	5
4.	Durabilité.....	5
C.	Le modèle BASE.....	6
1.	Basically Available (Fondamentalement Disponible)	6
2.	Soft-state (Etat souple).....	6
3.	Eventually Consistent (Cohérence Eventuelle)	6
III.	Modèles de données	6
A.	Le modèle de données de type clé-valeur	6
B.	Le modèle de données orienté colonne.....	7
C.	Le modèle de données de type document.....	7
D.	Le modèle de données de type graph	8
IV.	Installation et utilisation de MongoDB.....	9
A.	Définitions	9
B.	Modélisation.....	9
1.	La dénormalisation.....	9
2.	Les références	9

3.	Validations.....	11
C.	Installation.....	11
1.	En local.....	11
2.	Via MongoDB Atlas.....	11
D.	MongoDB Compass.....	12
E.	Manipulation de données dans le shell.....	12
1.	Insertion.....	12
2.	Récupération.....	13
3.	Mise à jour.....	15
4.	Suppression.....	17
5.	Opérations en masse.....	18
6.	Pipelines d'agrégations.....	18
V.	Node.js et MongoDB.....	20
A.	Mongoose.....	20
B.	Connexion.....	20
C.	Modéliser les données.....	21
D.	Validations.....	21
E.	Relations.....	23
F.	Interactions.....	24
G.	Middlewares.....	27
1.	Différents types de middlewares.....	27
2.	Mise en place des middlewares.....	28
H.	Requêtes avancées.....	29
I.	Sécurité et performances.....	30
1.	Les index.....	30
2.	Sécurité.....	31
3.	Performance.....	31

I. Introduction

A. Qu'est-ce qu'une base de données ?

Une base de données est un outil utilisé en informatique qui permet de stocker et d'organiser des données. L'objectif étant de les rendre accessible et d'en faciliter l'accès et la mise à jour.

B. Bases de données relationnelles ou non relationnelles

En développement web, nous avons le choix de travailler avec des bases de données relationnelles ou non relationnelles.

1. Base de données relationnelles

Dans ce système également nommé **SQL**, les données sont stockées dans des **tables** liées entre elles par des **relations**. Chaque table est composée de **colonnes** qui représentent des types de données différents comme un prénom, un nom, une adresse e-mail, etc. Les tables sont reliées entre elles via des **clés primaires et étrangères**.

On les utilise pour les projets qui ont besoin d'une **cohérence forte et d'une structure de données stable**.

2. Base de données non relationnelles

Aussi appelées **NoSQL**, les bases de données non relationnelles utilisent des structures bien différentes des bases de données relationnelles. On stocke cette fois les données sous forme de **documents, de graphes, de paires clé-valeur ou de colonnes**. Tout dépend du SGBD utilisé. Il n'y a pas de schéma fixe, ce qui permet de stocker des données non structurées ou semi-structurées facilement.

On les utilise pour les projets qui ont besoin d'une **grande évolutivité et flexibilité**.

3. Comparatif

	SQL	NoSQL
Modèle	Relationnel	Non-relationnel
Structure des tables	Structurées	Semi-structurées
Schéma	Strict	Dynamique
Transactions	ACID	BASE (mais support complet d'ACID avec MongoDB moderne)
Evolutivité	Verticale	Verticale et Horizontale
Flexibilité	Rigide	Flexible

4. Types d'évolutivité

a) *Evolutivité verticale*

Ce type d'évolutivité consiste à augmenter les ressources (mémoire, CPU, disque) d'un serveur pour en améliorer les performances. On peut par exemple ajouter plus de RAM afin de pouvoir gérer plus de requêtes en simultané. On peut améliorer les performances jusqu'à atteindre les capacités physiques de la machine.

b) *Evolutivité horizontale*

Ici, on va ajouter des serveurs supplémentaires pour améliorer les performances plutôt que d'ajouter des ressources au serveur existant. On va alors distribuer la charge de travail entre les serveurs. C'est plus flexible car on peut alors ajouter autant de serveurs que nécessaire pour répondre aux besoins de l'application.

II. Grands concepts

A. Le théorème CAP

Le théorème CAP est un théorème fondamental en informatique qui explique qu'il est impossible pour un système de garantir simultanément la cohérence des données, la disponibilité et la tolérance aux partitionnement. On ne peut garantir que deux de ces trois propriétés. Il est alors toujours nécessaire de faire des compromis entre cohérence, disponibilité et tolérance au partitionnement.

1. Consistency (Cohérence des données)

Toutes les copies distribuées dans un système doivent être identiques et conformes aux règles d'intégrité et de contrainte de la base de données. En pratique, toutes les lectures sur une donnée doivent retourner la dernière écriture ou la dernière version valide de cette donnée.

2. Availability (Disponibilité)

Chaque requête doit recevoir une réponse sans garantir que cette réponse soit forcément la dernière version des données disponibles.

3. Partition Tolerance (Tolérance au partitionnement)

Le système doit continuer de fonctionner même si la communication réseau entre les différents serveurs (les nœuds) est coupée.

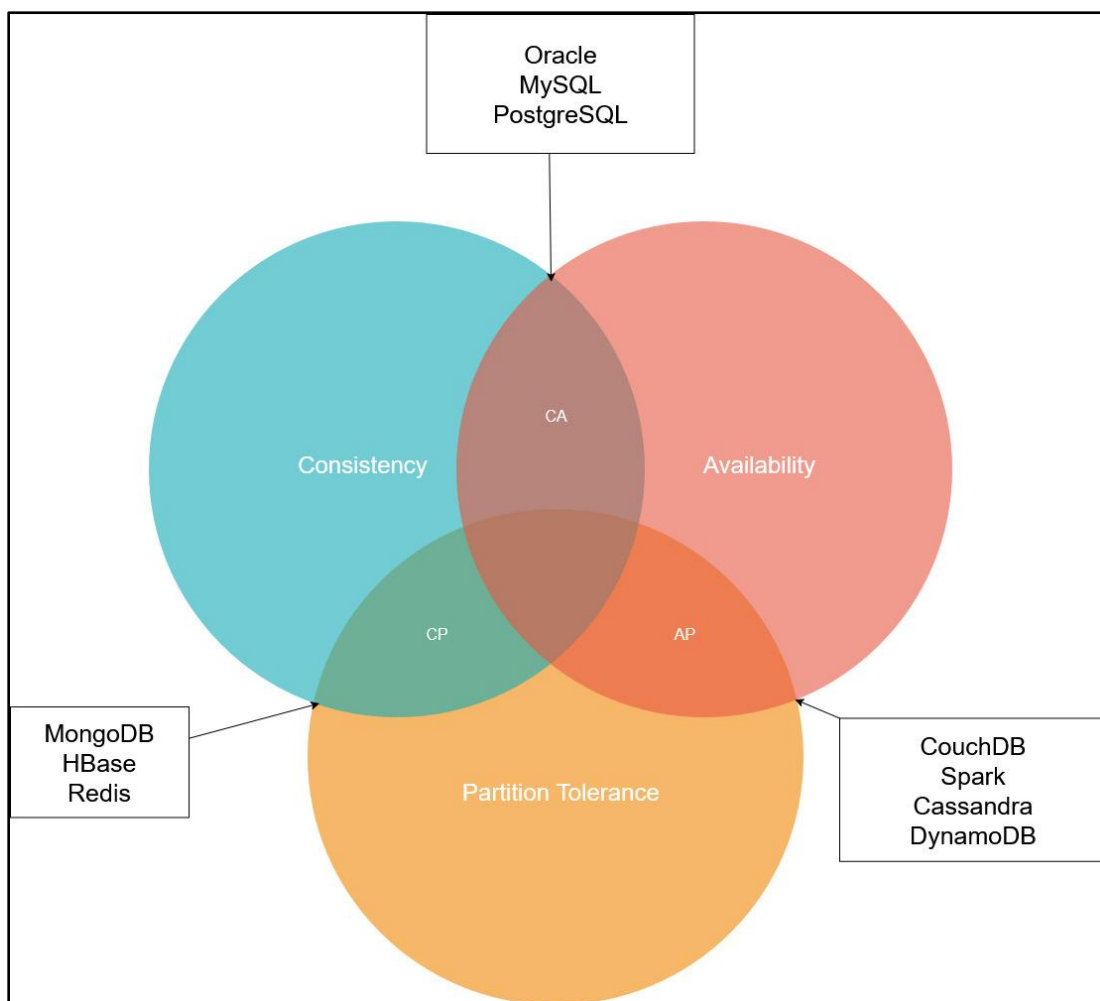


Figure 1: Schématisation du théorème CAP et des SGBD liés.

4. Les compromis

a) *Le couple CA (Consistency et Availability)*

Ici la cohérence des données est garantie, ainsi que la disponibilité du système. En pratique, toutes les lectures renvoient la dernière écriture tout en garantissant que le système est toujours disponible pour répondre aux demandes. En revanche, ce système ne supporte pas le partitionnement réseau : si une coupure survient entre les serveurs, le système ne peut plus fonctionner correctement.

b) *Le couple CP (Consistency et Partition Tolerance)*

Ici la cohérence des données est garantie, ainsi que la tolérance aux partitions. En pratique, toutes les lectures renvoient la dernière écriture, même en cas de partition réseau. Cependant, le système peut être temporairement indisponible s'il est partitionné en plusieurs sous-ensembles.

c) *Le couple AP (Availability et Partition Tolerance)*

Ici la disponibilité est garantie, ainsi que la tolérance aux partitions. En pratique, le système est toujours disponible pour répondre aux demandes même en cas de partitions ou de pannes. Cependant, les données renvoyées peuvent ne pas toujours être cohérentes.

B. Le modèle ACID

ACID représente un ensemble de propriétés qui garantissent la fiabilité et la cohérence des transactions dans une base de données relationnelle.

En NoSQL ces principes ne sont pas toujours garantis. Certaines bases de données NoSQL offrent cependant des fonctionnalités similaires pour garantir fiabilité et cohérence des données. Tout est question de compromis entre performances et fiabilité.

C'est le cas de [MongoDB](#) qui, depuis ses versions récentes, intègre et supporte pleinement les transactions ACID multi-documents.

1. Atomicité

Une transaction est atomique si elle est exécutée comme une seule unité indivisible. En pratique, cela signifie que toute modification apportée doit être exécutée avec succès ou complètement annulée en cas d'erreur. On ne doit pas avoir de transaction à moitié effectuée.

2. Cohérence

Toutes les données doivent respecter les règles d'intégrité et de contraintes définies par la base de données. Si ce n'est pas le cas, la base de données revient à son état antérieur.

3. Isolation

Chaque transaction doit être exécutée indépendamment des autres transactions qui peuvent être exécutées en même temps. Les modifications apportées par une transaction ne sont donc pas visibles par les autres tant qu'elles ne sont pas validées.

4. Durabilité

Toutes les modifications apportées à une base de données doivent être stockées de manière permanente et fiable. On cherche ici à s'assurer que les données ne seront pas perdues en cas de panne du système ou d'autres erreurs.

C. Le modèle BASE

BASE est un ensemble de principes qui guident la conception des bases de données NoSQL. Ils ont pour objectif d'offrir une haute disponibilité, une évolutivité horizontale et une performance élevée tout en faisant des compromis sur la garantie de la cohérence des données.

1. Basically Available (Fondamentalement Disponible)

On garantit une haute disponibilité permettant aux données d'être disponibles en permanence même en cas de panne ou d'erreur. Si certaines données sont temporairement indisponibles, le système restera, lui, dans l'ensemble disponible.

2. Soft-state (Etat souple)

L'état du système peut évoluer de manière fluide et transitoire, sans intervention de l'utilisateur, le temps que les données se synchronisent. Ceci permet de supporter des changements fréquents dans l'état des données et une évolutivité horizontale à grande échelle.

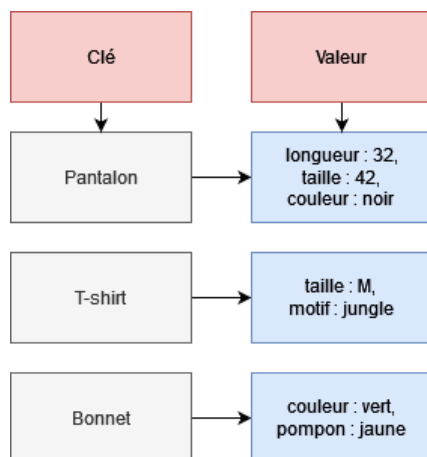
3. Eventually Consistent (Cohérence Eventuelle)

On vise ici une cohérence des données à long terme et non pas à chaque instant précis. Les modifications apportées à des données pouvant prendre plus de temps pour se propager sur l'ensemble du système, on peut avoir une incohérence temporaire. Cependant, le système finira par revenir à un état cohérent. Ce fonctionnement permet également une évolutivité à grande échelle car elle permet à plusieurs nœuds de fonctionner en toute autonomie et de regrouper les données ultérieurement.

III. Modèles de données

A. Le modèle de données de type clé-valeur

On associe ici une clé à une valeur. La clé sert d'identifiant unique pour la donnée. Il n'y a pas de schéma fixe, les données peuvent donc être de différents types (string, integer, json...) sans problème. On les utilise surtout pour le stockage de données temporaires ou pour un accès rapide avec des requêtes très simples. La plupart du temps, les données n'ont aucun lien entre elles.



Avantages : grande évolutivité horizontale, performances ultra-rapides, flexibilité, légèreté et simplicité de mise en œuvre.

Inconvénients : impossibilité d'effectuer des requêtes complexes sur le contenu des valeurs (on ne peut interroger la base que par sa clé exacte), pas de jointures natives et modélisation des relations très difficile.

Exemples : Redis, DynamoDB, Riak, Memcached.

Figure 2: Schéma du modèle de données de type clé-valeur

B. Le modèle de données orienté colonne

On retrouve ici la notion de tables et de colonnes comme pour les bases de données relationnelles. La différence réside dans le fait que le nombre de colonnes n'est pas fixe. On peut ajouter des colonnes dynamiquement et les valeurs nulles ont un coût de stockage nul (elles ne prennent pas de place). Les données sont stockées par colonnes et non pas par lignes. On l'utilise surtout dans le domaine du Big Data pour la gestion, le stockage et l'analyse de très grands volumes de données.

Avantages : haute performance en lecture, traitements d'agrégation faciles sur les colonnes (comptage, moyenne...), stockage très efficace, forte évolutivité horizontale, schéma flexible et calculs statistiques rapides.

Inconvénients : moins performant pour l'écriture, plus complexe à concevoir, peu adapté à la lecture d'une ligne entière de données spécifiques.

Exemples : HBase, Cassandra, Accumulo.

Row key	name	details:couleur	details:taille	details:longueur	details:motif	details:pompon
1	Pantalon	noir	42	32	null	null
2	T-shirt	null	M	null	jungle	null
3	Bonnet	vert	null	null	null	jaune

Figure 3: Schéma du modèle de données orienté colonne. Les cases « null » sont en réalité inexistantes.

Row key	name
1	Pantalon
2	T-shirt
3	Bonnet

Row key	couleur
1	noir
3	vert

Row key	taille
1	42
2	M

Figure 4: Autre représentation du modèle de données orienté colonne.

C. Le modèle de données de type document

Ce type repose sur le modèle de données de type clé-valeur. La valeur est alors ici un document de type BSON.

Petite mise au point :

JSON (ou *JavaScript Object Notation*) est un format standard d'échange de données qui permet de représenter des données structurées. Il est très similaire aux objets JavaScript, dont il s'inspire. Il a rapidement posé plusieurs problèmes dans l'utilisation des bases de données. Étant basé sur du texte, son analyse est lente. De plus, il prend beaucoup d'espace et ne prend en charge qu'un nombre réduit de types de données en base.

BSON (ou *Binary JSON*) a été inventé pour bénéficier des avantages du JSON en comblant ses défauts. Il encode directement les informations de type et de longueur, ce qui rend son analyse beaucoup plus rapide. Au fil du temps, de nouveaux types de données ont été ajoutés (les dates, etc.). On a aussi ajouté un certain nombre de distinctions comme la taille des entiers en octets ou le degré de précision des décimaux (nombre de chiffres après la virgule).

Dans ce modèle, chaque instance d'un objet est représentée par un **document** et chaque champ est représenté par une propriété de l'objet. Chaque document peut inclure des sous-documents. On appelle un ensemble de documents une **collection**. La clé unique est généralement nommée « id » ou « _id ».

```
[
  {
    "_id": "ObjectId(\"615bc6f93a373bd004b0831c\")",
    "name": "Pantalon",
    "details": {
      "longueur": "32",
      "taille": "42",
      "couleur": "noir"
    }
  },
  {
    "_id": "ObjectId(\"615bc71b3a373bd004b0831d\")",
    "name": "T-shirt",
    "details": {
      "taille": "M",
      "motif": "jungle"
    }
  },
  {
    "_id": "ObjectId(\"615bc7303a373bd004b0831e\")",
    "name": "Bonnet",
    "details": {
      "couleur": "vert",
      "pompon": "jaune"
    }
  }
]
```

Avantages : flexibilité (chaque document peut avoir sa propre structure), performances élevées, évolutivité horizontale, autonomie (toutes les informations peuvent être incluses directement dans le document de manière hiérarchique), richesse de la structure.

Inconvénients : pas de jointures natives classiques (difficulté à lier des données de plusieurs collections), requêtes complexes en raison de la flexibilité des documents, redondance des données (duplication).

Exemples : MongoDB, CouchDB, Couchbase.

Figure 5 : Schéma du modèle de données de type document en BSON.

D. Le modèle de données de type graph

Ce modèle repose sur le concept de graphes. C'est un ensemble de **nœuds** reliés entre eux par des **arêtes**. Chaque nœud est une représentation d'une entité du monde réel (une personne, un pays, un produit...). Chaque arête représente une relation entre deux entités (ex : tel produit est disponible dans telle ville).

Les requêtes se basent sur le parcours du graphe en suivant les arêtes et les nœuds qui répondent à la demande. Ces deux éléments peuvent posséder chacun leurs propres propriétés.

On les utilise beaucoup pour tout ce qui touche aux données en réseau : les réseaux sociaux, les moteurs de recommandation, les données géospatiales ou encore le web sémantique.

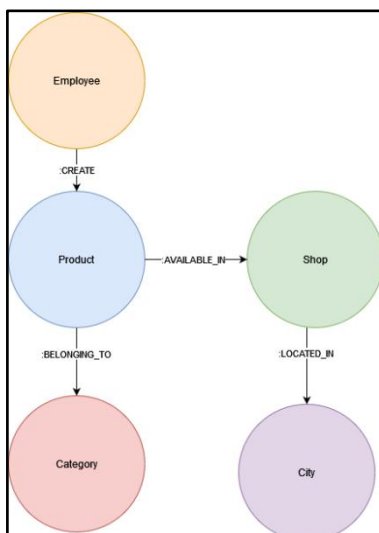


Figure 6 : Schéma du modèle de données de type graph.

Avantages : modélisation facile des relations multiples et complexes, requêtes de parcours ultra-rapides et grande flexibilité.

Inconvénients : difficulté à visualiser et comprendre la structure globale du graphe dans les cas très complexes, optimisation difficile des requêtes pour les bases de données massives, moins adapté aux données purement structurées.

Exemples : Neo4j, OrientDB, ArangoDB, InfoGrid.

ACTIVITE 1 : QCM

https://docs.google.com/forms/d/e/1FAIpQLSfn7SffXvkJwea9JErCxzla_7tHOpDzD6FqmA0-zuS8hluZg/viewform?usp=share_link

IV. Installation et utilisation de MongoDB

A. Définitions

MongoDB est un **SGBD NoSQL orienté document**. Il stocke les données en BSON et utilise une architecture distribuée (répartition sur plusieurs nœuds/machines). On l'utilise pour stocker des données semi-structurées ou non structurées. C'est un SGDB très performant qui offre une grande disponibilité, une évolutivité horizontale et une distribution géographique.

C'est un SGDB open-source mais qui propose également l'achat de licences commerciales pour l'utilisation professionnelle.

B. Modélisation

1. La dénormalisation

La dénormalisation consiste à stocker les données de manière redondante au sein de plusieurs documents. Ceci permet d'améliorer les performances en lecture, car toutes les informations nécessaires sont directement disponibles au sein d'un même document. Il n'est plus nécessaire de multiplier les requêtes ou d'utiliser des jointures complexes.

En contrepartie, les écritures et les mises à jour deviennent plus lourdes, car il faut modifier la donnée partout où elle a été dupliquée.



Figure 7: Un document dénormalisé (source : <https://www.mongodb.com/docs/manual/data-modeling/embedding/>)

Liens utiles :

- <https://www.mongodb.com/docs/manual/data-modeling/embedding/>
- <https://www.mongodb.com/docs/manual/tutorial/model-embedded-one-to-many-relationships-between-documents/>

2. Les références

Les références permettent de normaliser. Ici les documents sont liés entre eux par une clé unique appelée « `_id` » du type « `ObjectId` ». C'est un système moins performant mais qui permet d'éviter la redondance des données.

C'est un système moins performant en lecture car il nécessite de faire des requêtes supplémentaires (ou d'utiliser *\$lookup*), mais il permet d'éviter totalement la redondance des données et facilite les mises à jour.

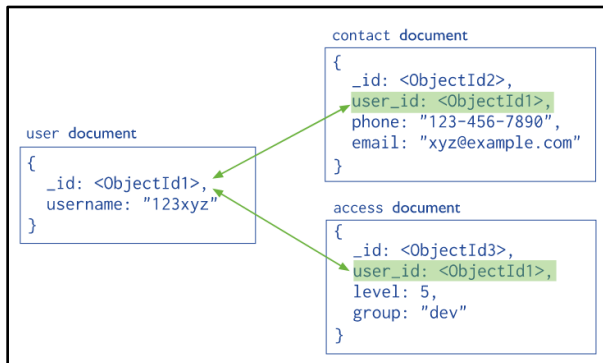


Figure 8: Illustration de la normalisation avec les références (source : <https://www.mongodb.com/docs/manual/data-modeling/referencing/>)

Liens utiles :

- <https://www.mongodb.com/docs/manual/data-modeling/referencing/>
- <https://www.mongodb.com/docs/manual/tutorial/model-referenced-one-to-many-relationships-between-documents/>

ACTIVITE 2 : Dénormaliser

On a deux documents :

- **User**

```

{
  "_id": ObjectId("60a1fbf9940c3e0f3c3e8dd3"),
  "name": "John Doe",
  "email": "john.doe@example.com",
  "age": 30
}
    
```

- **Post**

```

{
  "_id": ObjectId("60a1fc4b940c3e0f3c3e8dd4"),
  "title": "My first post",
  "content": "Lorem ipsum dolor sit amet...",
  "user_id": ObjectId("60a1fbf9940c3e0f3c3e8dd3")
}
    
```

Dénormaliser « à la main » le document Post et me l'envoyer.

3. Validations

Il est possible de [valider les données](#) avant leur insertion en base de données afin de s'assurer qu'elles respectent un certain nombre de règles (champs obligatoires, types de données, formats, etc.). Les schémas de validation se définissent directement au niveau de la collection.

Si l'on ajoute un schéma de validation à une collection qui contient déjà des données, deux règles s'appliquent :

- Les documents nouvellement insérés ou mis à jour sont systématiquement vérifiés.
- Les documents déjà existants et non modifiés ne sont pas revérifiés automatiquement (ils restent présents même s'ils ne respectent pas le nouveau schéma). Au besoin, on peut utiliser la commande [db.collection.validate\(\)](#) pour scanner la collection et lister les anciens documents qui ne respectent pas les nouvelles règles.

En cas d'erreur lors d'une insertion, le document est rejeté et n'est pas ajouté à la collection.

La validation s'active en définissant un **objet validator** sur la collection. Pour rédiger les règles de ce validateur, deux syntaxes sont possibles :

- En utilisant le JSON Schema (recommandé) → Permet de décrire précisément la structure attendue via l'opérateur `$jsonSchema`
<https://www.mongodb.com/docs/manual/core/schema-validation/specify-json-schema/>
- En utilisant les opérateurs de requête classiques → Permet de spécifier des règles basées sur des expressions de correspondance
Règles → <https://www.mongodb.com/docs/manual/core/schema-validation/specify-query-expression-rules/>
Opérateurs → <https://www.mongodb.com/docs/manual/reference/operator/query/>

C. Installation

1. En local

Pour MacOS : <https://www.mongodb.com/docs/manual/tutorial/install-mongodb-on-os-x/>

Pour Linux : <https://www.mongodb.com/docs/manual/administration/install-on-linux/>

Pour Windows : <https://www.mongodb.com/docs/manual/tutorial/install-mongodb-on-windows/>

Pour Docker : <https://www.mongodb.com/docs/manual/tutorial/install-mongodb-community-with-docker/>

ACTIVITE 3 : Installation

Installer MongoDB en local sur la machine et m'envoyer la capture d'écran d'une commande permettant de vérifier que l'installation est bien effective.

2. Via MongoDB Atlas

MongoDB Atlas est un service de base de données Cloud qui offre une solution managée pour stocker et gérer des données avec MongoDB. C'est un service facile à utiliser, sécurisé et hautement disponible, permettant de s'affranchir de toute la configuration matérielle et de la maintenance du serveur. Il intègre des fonctionnalités avancées telles que la sauvegarde automatique, la reprise après sinistre, l'évolutivité horizontale, la gestion automatique des performances et une sécurité renforcée.

Il permet également de choisir l'infrastructure Cloud chez qui stocker nos données parmi les trois grands fournisseurs du marché : AWS, Google Cloud Platform ou Microsoft Azure.

Démarrer avec MongoDB Atlas → <https://www.mongodb.com/docs/atlas/getting-started/>

ACTIVITE 4 : Création du compte

Créer un compte MongoDB Atlas (via l'UI et non la CLI) et envoyer une capture d'écran du compte validé.

D. MongoDB Compass

MongoDB Compass est un outil de visualisation et d'exploration de données pour MongoDB. C'est une interface graphique similaire à phpMyAdmin pour le SGBD MySQL. On peut ainsi facilement interagir avec les données stockées dans des bases de données locales ou des clusters MongoDB Atlas.

C'est un logiciel relativement intuitif et facile à utiliser. Il permet d'exécuter graphiquement l'ensemble des opérations CRUD (Création, Lecture, Mise à jour, Suppression). On peut également y analyser aisément la structure des documents (les schémas de données), gérer les index et suivre les statistiques de performance. De plus, le *shell mongosh* y est directement intégré.

Téléchargement → <https://www.mongodb.com/try/download/compass>

ACTIVITE 5 : Installation du logiciel et connexion

Installer MongoDB Compass, le connecter à une base de données locale ou à un cluster Atlas. M'envoyer une capture d'écran de l'interface une fois la connexion réussie.

E. Manipulation de données dans le shell

1. Insertion

a) Insérer un seul document

La méthode `db.collection.insertOne()` permet d'ajouter un unique document au format BSON dans une collection. Si la collection n'existe pas encore, MongoDB la crée automatiquement.

```
db.movies.insertOne({
  title: "The Favourite",
  genres: [ "Drama", "History" ],
  runtime: 121,
  rated: "R",
  year: 2018,
  directors: [ "Yorgos Lanthimos" ],
  cast: [ "Olivia Colman", "Emma Stone", "Rachel Weisz" ],
  type: "movie"
})
```

ACTIVITE 6 : Créer les catégories

1. Créer (ou basculer sur) une base de données nommée « e-commerce » (<https://www.mongodb.com/resources/products/fundamentals/create-database>)

Dans MongoDB, une base de données n'est réellement créée qu'au moment de la première insertion de données.

2. Dans cette base de données et au sein d'une collection nommée « category », ajouter **une à une** les [catégories](#).

3. M'envoyer une capture d'écran des commandes.

b) Insérer plusieurs documents

La méthode [`db.collection.insertMany\(\)`](#) permet d'ajouter un tableau de documents au format BSON dans une collection en une seule opération. Les documents doivent obligatoirement être enveloppés dans des crochets [].

```
db.movies.insertMany([
  {
    title: "Jurassic World: Fallen Kingdom",
    genres: [ "Action", "Sci-Fi" ],
    runtime: 130,
    rated: "PG-13",
    year: 2018,
    directors: [ "J. A. Bayona" ],
    cast: [ "Chris Pratt", "Bryce Dallas Howard", "Rafe Spall" ],
    type: "movie"
  },
  {
    title: "Tag",
    genres: [ "Comedy", "Action" ],
    runtime: 105,
    rated: "R",
    year: 2018,
    directors: [ "Jeff Tomsic" ],
    cast: [ "Annabelle Wallis", "Jeremy Renner", "Jon Hamm" ],
    type: "movie"
  }
])
```

ACTIVITE 7 : Ajouter les clients et les produits

1. Dans une collection nommée « product », ajouter tous les [produits](#).

Pour lier chaque produit à sa catégorie, utiliser son « `_id` » sous forme d'`ObjectId`.

→ <https://www.mongodb.com/docs/manual/reference/method/ObjectId/>

2. Ajouter les [clients](#) dans une collection nommée « client », puis les [commandes](#) dans une collection nommée « order ».

3. M'envoyer une capture d'écran des commandes.

Pour aller plus loin : comportements à l'insertion

<https://www.mongodb.com/docs/manual/tutorial/insert-documents/#insert-behavior>

2. Récupération

a) Tout récupérer

La méthode [`db.collection.find\(\)`](#) permet de chercher et de lire des documents dans une collection. Pour récupérer l'intégralité des documents sans aucun filtre, on passe un objet vide {} en paramètre (ou on laisse les parenthèses vides ()).

```
db.movies.find({}) // ou simplement : db.movies.find()
```

ACTIVITE 8 : Retrouver les commandes

1. Retrouver l'ensemble des commandes passées sur le site.

2. M'envoyer une capture d'écran de la commande et du résultat.

b) Retrouver avec une condition simple

Pour filtrer les documents et ne récupérer que ceux qui correspondent à un critère précis, on passe un objet contenant la condition sous la forme `{ champ: valeur }`.

```
db.movies.find( { runtime: 130 } )
```

ACTIVITE 9 : Retrouver les commandes d'un client

1. Retrouver l'ensemble des commandes passées par le client « M. Dupont »
2. M'envoyer une capture d'écran de la commande et du résultat.

ACTIVITE 10 : Retrouver les commandes contenant un certain produit

1. Retrouver l'ensemble des commandes passées sur le site contenant le produit « Harry Potter... »
2. M'envoyer une capture d'écran de la commande et du résultat.

c) Retrouver un seul document avec une condition simple

La méthode `db.collection.findOne()` fonctionne comme `find()`, mais elle ne retourne que le tout premier document qui correspond au critère de recherche, même si plusieurs documents valident la condition. C'est idéal pour chercher un élément unique (comme un identifiant).

```
db.movies.findOne( { _id: ObjectId("573a1390f29313caabcd41e1") } )
```

ACTIVITE 11 : Retrouver un produit spécifique

1. Retrouver le produit « MacBook Air »
2. M'envoyer une capture d'écran de la commande et du résultat.

d) Retrouver selon plusieurs conditions

Il est possible de combiner plusieurs critères de recherche au sein d'un même objet. Par défaut, séparer les critères par une virgule effectue une opération logique « ET » (AND). On peut également y intégrer des [opérateurs de comparaison](#) (comme `$lt` pour « inférieur à »).

```
db.movies.find( { year: 2018, runtime: { $lt: 120 } } )
```

ACTIVITE 12 : Retrouver tous les produits selon plusieurs conditions

1. Retrouver tous les produits de la catégorie des livres et dont le prix est strictement inférieur à 50 €
2. M'envoyer une capture d'écran de la commande et du résultat.

e) Retrouver avec l'opérateur IN :

L'opérateur `$in` permet de rechercher des documents dont la valeur d'un champ correspond à l'un des éléments spécifiés dans un tableau. Cela équivaut à une suite de conditions « OU » (OR).

```
db.movies.find( { runtime: { $in: [ "130", "105" ] } } )
```

ACTIVITE 13 : Retrouver tous les produits selon des prix définis

1. Retrouver tous les produits dont le prix est égal à 69.99 € ou 79.99 €
2. M'envoyer une capture d'écran de la commande et du résultat.

f) *Retrouver avec l'opérateur OR*

L'opérateur \$or permet d'effectuer un « **OU** ». Il permet de récupérer les documents qui respectent au moins l'une des conditions fournies. Contrairement aux filtres classiques, sa syntaxe prend un tableau de critères.

```
db.movies.find({
  $or: [
    { type: "short film" },
    { runtime: { $lt: 60 } }
  ]
})
```

ACTIVITE 14 : Retrouver des produits selon leur catégorie ou leur prix

1. Retrouver tous les produits dont le prix est supérieur ou égal à 100 € ou qui appartiennent à la catégorie « Electronics »
2. M'envoyer une capture d'écran de la commande et du résultat.

g) *Combiner les opérateurs logiques (AND et OR)*

La virgule sert à faire un **ET**. On peut donc facilement l'associer à un bloc \$or (**OU**) pour créer des filtres plus précis.

```
db.movies.find({
  year: 2023,
  $or: [
    { type: "short film" },
    { runtime: { $lt: 60 } }
  ]
})
```

ACTIVITE 15 : Retrouver les commandes d'un client selon les caractéristiques des produits.

1. Retrouver toutes les commandes du client « M. Dupont » qui contiennent un produit de type « iPhone 12 » OU dont la quantité est supérieure ou égale à 2.
2. M'envoyer une capture d'écran de la commande et du résultat.

Pour aller plus loin : Les opérateurs de requête

<https://www.mongodb.com/docs/manual/reference/operator/query/>

Pour aller plus loin : Les requêtes plus complexes

<https://www.mongodb.com/docs/manual/tutorial/query-documents/#additional-query-tutorials>

3. Mise à jour

a) *Mettre à jour un document*

La méthode [db.collection.updateOne\(\)](#) permet de modifier le premier document qui correspond au filtre de recherche. Elle prend principalement deux arguments : le filtre pour trouver le document, et les modifications à appliquer (en utilisant l'opérateur \$set).

Syntaxe globale :

```
db.collection.updateOne(<filter>, <update>, <options>)
```

Exemple d'utilisation :

```
db.movies.updateOne(
  { title: "The Favourite" },
  { $set: { runtime: 125 } }
)
```

ACTIVITE 16 : Modifier un tarif

1. Passer le prix du « MacBook Air » à 1249.99 €.
2. M'envoyer une capture d'écran de la commande et de la confirmation du terminal.

b) Mettre à jour plusieurs documents

La méthode [db.collection.updateMany\(\)](#) fonctionne exactement comme `updateOne()`, mais elle applique les modifications à tous les documents qui correspondent au filtre de recherche, et non pas uniquement au premier trouvé.

Syntaxe globale :

```
db.collection.updateMany(<filter>, <update>, <options>)
```

Exemple d'utilisation :

```
db.movies.updateMany(
  { "runtime": { $lt: 60 } },
  {
    $set: { "type": "short movie" }
  }
)
```

ACTIVITE 17 : Ajouter un statut aux clients

1. Ajouter à l'ensemble des clients un champ « status » avec pour valeur « active ».
2. M'envoyer une capture d'écran de la commande et de la confirmation du terminal.

c) Remplacer complètement le document

La méthode [db.collection.replaceOne\(\)](#) permet de remplacer l'intégralité d'un document par un nouveau document, tout en conservant son identifiant unique « `_id` ». Contrairement aux méthodes de mise à jour, on n'utilise pas l'opérateur `$set` ici : le contenu fourni écrasera totalement l'ancien.

Syntaxe globale :

```
db.collection.replaceOne(<filter>, <newDocument>, <options>)
```

Exemple d'utilisation :

```
db.movies.replaceOne(
  { "title" : "Jurassic World: Fallen Kingdom" },
  { "name" : "Jurassic World: Fallen Kingdom", "status" : "archived" }
);
```

ACTIVITE 18 : Remplacer complètement une catégorie

1. Remplacer l'intégralité de la catégorie « Clothing » par le document suivant :

```
{
  "name" : "outfit"
}
```

2. M'envoyer une capture d'écran de la commande et de la confirmation du terminal.

Pour aller plus loin : Les opérateurs de mise à jour

<https://www.mongodb.com/docs/manual/reference/operator/update/>

Pour aller plus loin : Des méthodes pour retrouver et mettre à jour le document

Par défaut, les méthodes `updateOne()`, `updateMany()` et `replaceOne()` ne renvoient pas le document modifié, mais uniquement un rapport d'exécution (ex: nombre de documents correspondants, nombre de documents modifiés).

Lorsqu'on a besoin de récupérer directement le contenu du document au moment où on le modifie, MongoDB propose deux méthodes spécifiques [`findOneAndUpdate\(\)`](#) et [`findOneAndReplace\(\)`](#).

Ces méthodes acceptent une option très importante nommée [`returnDocument`](#) (ou [`returnNewDocument`](#)) qui permet de choisir ce que la commande doit renvoyer dans le terminal.

4. Suppression

a) Supprimer un seul document

La méthode [`db.collection.deleteOne\(\)`](#) permet de supprimer le premier document qui correspond au filtre de recherche spécifié.

```
db.movies.deleteOne( { status: "archived" } )
```

ACTIVITE 19 : Supprimer un produit

1. Supprimer le produit « Nike Air Max 270 ».
2. M'envoyer une capture d'écran de la commande et de la confirmation du terminal.

b) Supprimer tous les documents selon une condition

La méthode [`db.collection.deleteMany\(\)`](#) fonctionne comme `deleteOne()`, mais elle supprime tous les documents qui correspondent au filtre de recherche spécifié.

```
db.movies.deleteMany({ year : { $lt: 2018 } })
```

ACTIVITE 20 : Supprimer tous les produits d'une catégorie

1. Supprimer tous les produits de la catégorie « Books ».
2. M'envoyer une capture d'écran de la commande et de la confirmation du terminal.

c) Supprimer tous les documents

Pour vider entièrement une collection sans pour autant supprimer la collection elle-même, on utilise la méthode [`db.collection.deleteMany\(\)`](#) en lui passant un filtre totalement vide {}.

```
db.movies.deleteMany({})
```

ACTIVITE 21 : Supprimer toutes les commandes

1. Supprimer toutes les commandes sans exception.
2. M'envoyer une capture d'écran de la commande et de la confirmation du terminal.

Pour aller plus loin : Retrouver puis supprimer le document

Par défaut, les méthodes `deleteOne()` et `deleteMany()` ne renvoient pas le document supprimé, mais uniquement un rapport d'exécution (ex : nombre de documents supprimés).

Lorsqu'on a besoin de récupérer directement le contenu du document au moment où on le supprime, MongoDB propose une méthode spécifique [`findOneAndDelete\(\)`](#).

Ceci est utile si on a besoin de récupérer les données avant de les supprimer.

5. Opérations en masse

Il est possible de **réaliser plusieurs opérations d'écriture en une seule commande** grâce à la méthode [`bulkWrite\(\)`](#).

Cette méthode supporte les actions suivantes :

- `insertOne`
- `updateOne`
- `updateMany`
- `replaceOne`
- `deleteOne`
- `deleteMany`

Les opérations envoyées au `bulkWrite()` peuvent être exécutées selon deux modes :

- **Ordonnées (Ordered)** : les opérations sont exécutées strictement les unes à la suite des autres dans l'ordre du tableau. En cas d'erreur sur une action, MongoDB stoppe immédiatement le traitement et retourne l'erreur sans poursuivre la liste d'attente. C'est le comportement appliqué par défaut.

<https://www.mongodb.com/docs/manual/reference/method/db.collection.bulkWrite/#ordered-bulk-write-example>

- **Non ordonnées (Unordered)** : MongoDB peut exécuter les opérations en parallèle, sans garantie sur l'ordre final d'exécution. En cas d'erreur sur une opération, MongoDB ignore le problème et continue le traitement de toutes les autres actions restantes.

<https://www.mongodb.com/docs/manual/reference/method/db.collection.bulkWrite/#unordered-bulk-write-example>

6. Pipelines d'agrégations

Un [`pipeline d'agrégation`](#) est une séquence ordonnée d'opérations qui permettent de filtrer, transformer, trier et grouper les données d'une collection.

Le pipeline est construit en plusieurs étapes successives (appelées *stages*). Chaque étape prend en entrée le résultat généré par l'étape précédente pour affiner ou transformer les données jusqu'au résultat final.

Exemple d'utilisation :

```
db.movies.aggregate( [
  {
    $match: { year: 2018 }
  },
  {
    $group: { _id: "$type", totalDuration: { $sum: "$runtime" } }
  },
  { $project: { _id: 0, type: "$_id", totalDuration: 1 } }
] )
```

On cherche ici à trouver tous les films de 2018 et à retourner le résultat sous la forme d'objets avec le type et le nombre total de minutes de film pour chaque catégorie.

Résultat :

```
< {
  totalDuration: 60,
  type: 'short movie'
}
{
  totalDuration: 356,
  type: 'movie'
}
```

Figure 9: On affiche la durée totale de visionnage par type de contenu uniquement pour l'année 2018.

Il est également possible d'utiliser ces pipelines d'agrégations lors d'une opération de mise à jour (*update*) → <https://www.mongodb.com/docs/manual/tutorial/update-documents-with-aggregation-pipeline/>

Pour aller plus loin : Les états et opérateurs d'agrégation

Les états : <https://www.mongodb.com/docs/manual/reference/operator/aggregation-pipeline/>

Les opérateurs : <https://www.mongodb.com/docs/manual/reference/operator/aggregation/>

ACTIVITE 22 : Récupérer les produits avec les catégories associées

1. Récupérer les produits de la catégorie « Electronics » ainsi que les données pour la catégorie liée à chaque élément.

On utilisera une jointure via un pipeline d'agrégation avec l'opérateur [*\\$lookup*](#).

2. M'envoyer une capture d'écran de la commande et du résultat.

ACTIVITE 23 : QCM

https://docs.google.com/forms/d/e/1FAIpQLSfLylzAtlw_mngcRm_2zMNSJOftxkk6vpnhAZVFr50YJcU50Q/viewform?usp=sf_link

V. Node.js et MongoDB

A. Mongoose

[Mongoose](#) est une bibliothèque de programmation orientée objet en JavaScript, appelée un **ODM** (*Object Data Modeling*). Elle permet de créer une connexion entre l'environnement Node.js et la base de données MongoDB.

Fonctionnalités principales :

- **Création de modèles** : permet de définir la structure de chaque collection en concevant des schémas contenant des propriétés, des types de données et des règles de validation.
- **Validation de données** : permet de définir des contraintes strictes pour chaque propriété (valeur minimale, valeur maximale, format d'email, expressions régulières...). Il est également possible de fournir des messages d'erreurs personnalisés.
- **Création et utilisation de middleware** : permet d'intercepter et de modifier les opérations sur les requêtes (par exemple, exécuter une fonction juste avant la sauvegarde d'un document).
- **Syntaxe simplifiée** : fournit une couche d'abstraction qui simplifie l'écriture des requêtes complexes comme les conditions multiples, le tri, la pagination ou encore les agrégations.
- **Gestion des relations** : facilite la gestion des relations entre les collections en utilisant des systèmes de références ou de sous-documents intégrés. Mongoose fournit notamment des fonctionnalités avancées comme *populate()*.

B. Connexion

La connexion à la base de données s'effectue en asynchrone à l'aide de la fonction [mongoose.connect\(\)](#).

Exemple de mise en œuvre :

```
const mongoose = require('mongoose');
require('dotenv').config();

mongoose.connect(process.env.MONGO_URI, {
  ssl: process.env.MONGO_SSL
})
.then(() => console.log('MongoDB connected !'))
.catch(() => console.log('Error with MongoDB connection'));
```

L'utilisation de variables d'environnement (*process.env*) via la bibliothèque [dotenv](#) permet d'isoler l'URI de connexion et les options de sécurité (comme l'activation du SSL/TLS) afin de ne pas exposer de données sensibles dans le code source.

C. Modéliser les données

a) Créer un schéma

Un schéma définit la structure des documents au sein d'une collection (champs, types de données, validations).

Exemple :

```
// app/models/user.js
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  lastname: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
});
```

b) Créer le modèle

Le modèle est une classe basée sur le schéma. Il fournit l'interface pour interagir avec la base de données (créer, lire, modifier, supprimer des documents).

Exemple :

```
const User = mongoose.model("User", userSchema);
```

c) Types de données

Mongoose accepte la plupart des [types](#) JavaScript et MongoDB standards (ex : String, Number, Date, Boolean, Array, ObjectId...). Il suffit de déclarer le type souhaité via la propriété « type ».

D. Validations

a) Validateurs intégrés

Mongoose propose un ensemble de validateurs prédéfinis pour contrôler la validité des données avant leur insertion en base de données.

- [Rendre un champ obligatoire](#)
- [Pour les nombres](#)
- [Pour les chaînes de caractères](#)
- Etc.

Exemple de mise en œuvre :

```
const breakfastSchema = new Schema({
  eggs: {
    type: Number,
    min: [6, 'Too few eggs'],
    max: 12
  },
  bacon: {
    type: Number,
    required: [true, 'Why no bacon?']
  },
  drink: {
    type: String,
    enum: ['Coffee', 'Tea'],
  }
});
```

b) Messages d'erreurs personnalisés

Il est possible de [personnaliser les messages d'erreurs](#) renvoyés lors d'un échec de validation en utilisant une syntaxe sous forme de tableau ou d'objet. Le message personnalisé est alors placé en deuxième valeur de la propriété de validation.

Mongoose permet également d'utiliser le mot-clé {VALUE} pour injecter dynamiquement la valeur erronée saisie par l'utilisateur directement dans le message d'erreur.

Exemple de mise en œuvre :

```
const breakfastSchema = new Schema({
  eggs: {
    type: Number,
    min: [6, 'Must be at least 6, got {VALUE}'],
    max: 12
  },
  drink: {
    type: String,
    enum: {
      values: ['Coffee', 'Tea'],
      message: '{VALUE} is not supported'
    }
  }
});
```

c) L'option « unique »

⚠ Attention : l'option unique n'est pas un validateur !

Elle est utilisée uniquement pour indiquer à MongoDB de créer un index unique sur la collection en base de données. Par conséquent, la validation n'est pas exécutée par Mongoose en amont du traitement de la requête. Si une valeur dupliquée est insérée, c'est la base de données MongoDB qui renverra une erreur d'indexation brute et non Mongoose via un mécanisme de validation d'erreur standard.

<https://mongoosejs.com/docs/validation.html#the-unique-option-is-not-a-validator>

Pour intercepter proprement ce cas et le traiter comme une erreur de validation classique avec un message personnalisé, on utilise généralement le plugin suivant en complément :

<https://www.npmjs.com/package/mongoose-unique-validator>

d) Validateur personnalisé

Il est également possible de créer des règles de validation sur mesure en utilisant la propriété [validate](#).

Exemple pour valider le format d'une adresse e-mail :

```
const userSchema = new Schema({
  email: {
    type: String,
    required: true,
    unique: true,
    validate: {
      validator: function(email) {
        const emailRegex = /^[^\w-\.\.]+\@([\w-]+\.)+[\w-]{2,4}$/;
        return emailRegex.test(email);
      },
      message: 'Veuillez fournir une adresse e-mail valide'
    }
  }
});
```

E. Relations

a) *Sous-documents*

Les sous-documents sont des documents imbriqués à l'intérieur d'un document principal. Dans Mongoose, ils possèdent leur propre schéma et sont intégrés directement dans les champs du schéma parent.

Exemple au niveau des schémas :

```
const mongoose = require('mongoose');

const addressSchema = new mongoose.Schema({
  city: String,
  zip_code: String,
  street: String
});

const userSchema = new mongoose.Schema({
  name: String,
  addresses: [addressSchema] // Tableau contenant des sous-documents
});

const User = mongoose.model('User', userSchema);
```

Exemple au niveau de l'ajout :

```
const user = new User({
  name: 'John Doe',
  addresses: []
});

// Ajout d'une adresse dans le tableau du sous-document
user.addresses.push({
  city: 'Paris',
  zip_code: '75001',
  street: 'Rue de Rivoli'
});

await user.save();
```

Exemple au niveau de la mise à jour :

```
const user = await User.findOne({ name: 'John Doe' });

// Utilisation de la méthode .id() pour retrouver le sous-document par son _id
user.addresses.id(_id).city = 'Lyon';

await user.save();
```

Exemple au niveau de la suppression :

```
const user = await User.findOne({ name: 'John Doe' });

// Suppression du sous-document ciblé par son _id
user.addresses.id(_id).deleteOne();

await user.save();
```

b) Références

Pour faire référence à un document stocké dans une autre collection (système similaire aux clés étrangères en SQL), il faut utiliser le type `mongoose.ObjectId` combiné au paramètre [ref](#).

Exemple des schémas pour lier un utilisateur à un article :

```
const mongoose = require('mongoose');

// Modèle User
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true },
});

const User = mongoose.model('User', userSchema);

// Modèle Post
const postSchema = new mongoose.Schema({
  title: { type: String, required: true },
  content: { type: String, required: true },
  author: { type: mongoose.ObjectId, ref: User },
});

const Post = mongoose.model('Post', postSchema);
```

Exemple sur une création :

```
const user = await User.findById(userId);

const post = new Post({
  title: 'Mon premier post',
  content: 'Contenu du post',
  author: user._id
});

await post.save();
```

F. Interactions

a) Création

Avec la méthode [save\(\)](#) :

```
const user = new User({ name: 'Will Riker' });
await user.save();
```

S'utilise sur une instance de modèle (un document configuré au préalable).

Avec la méthode [create\(\)](#) :

```
await Character.create([{ name: 'Will Riker' }, { name: 'Geordi LaForge' }]);
```

Méthode de commodité permettant de créer et de sauvegarder directement un ou plusieurs documents en une seule commande.

Avec la méthode [insertMany\(\)](#) :

```
await Character.insertMany([{ name: 'Will Riker' }, { name: 'Geordi LaForge' }]);
```

Idéale pour insérer de larges volumes de données d'un seul coup de manière très performante.

Pour aller plus loin : Différences entre `save`, `create`, `insertOne` et `insertMany`

`save` est une méthode propre à Mongoose qui s'exécute sur une instance de document préalablement créée. Elle applique strictement les validations du schéma et déclenche l'intégralité des middlewares configurés (comme les hooks *pre-save* ou *post-save*). C'est la méthode idéale lorsque l'on a besoin d'un contrôle total et d'une personnalisation des données (par exemple, pour hacher un mot de passe) avant leur enregistrement.

`create` est un raccourci fourni par Mongoose qui combine la création de l'instance et sa sauvegarde en une seule étape. Elle peut accepter un seul objet ou un tableau d'objets. En arrière-plan, Mongoose va instancier chaque document et appeler la méthode `save` individuellement pour chacun d'eux. Cela signifie que `create` garantit également l'exécution complète des validations de schéma et des middlewares, tout en offrant une syntaxe plus compacte.

`insertMany` est une méthode optimisée pour les insertions en masse. Contrairement aux deux précédentes, elle envoie une seule commande réseau brute contenant tous les documents directement à la base de données, ce qui la rend extrêmement performante pour l'import de gros volumes de données ou le *seeding*. Mongoose applique tout de même les validations de schéma sur les éléments avant l'envoi. En revanche, les middlewares liés au document individuel (comme *pre('save')* ou *post('save')*) ne sont pas déclenchés, puisqu'aucune instance n'est réellement sauvegardée une par une. Mongoose expose toutefois des hooks dédiés, *pre('insertMany')* et *post('insertMany')*, qui s'exécutent bien autour de l'opération globale.

`insertOne` est un raccourci Mongoose qui instancie le document puis appelle `save` dessus. Les validations de schéma et les hooks *pre-save/post-save* sont bien exécutés. Sa particularité : elle accepte aussi bien un objet brut qu'un document Mongoose déjà instancié, sans le ré-instancier inutilement, contrairement à `create` qui, lui, instancie toujours un nouveau document. En contrepartie, `insertOne` ne prend qu'un seul document à la fois (pas de tableau).

b) Lecture

Avec la méthode `find()` :

```
// Récupérer tous les documents
await MyModel.find({});

// Récupérer tous les documents dont le nom est 'john' et l'âge est supérieur ou égal à 18 ans
await MyModel.find({ name: 'john', age: { $gte: 18 } }).exec();
```

Permet de récupérer un tableau de documents correspondant aux critères de recherche. Si aucun critère n'est fourni, tous les documents sont retournés.

Avec la méthode `findById()` :

```
// Récupérer un document par son identifiant unique
await Adventure.findById(id).exec();
```

Permet de récupérer un unique document directement via son identifiant « `_id` ». Retourne `null` si aucun document ne correspond.

Avec la méthode `findOne()` :

```
// Récupérer le premier document dont le pays est 'Croatia'
await Adventure.findOne({ country: 'Croatia' }).exec();
```

Permet de récupérer le premier document qui correspond aux critères de recherche. Retourne `null` si aucun document ne correspond.

Les options de requêtes :

```
// Limite de 10 et offset de 5 (Pagination)
const users = await MyModel.find().limit(10).skip(5).exec();

// On inclut les champs name et email mais on exclut _id (Projection)
const users = await MyModel.find().select('name email -_id').exec();
// On trie par nom ascendant (Tri)
const users = await MyModel.find().sort({ name: 'asc' }).exec();
```

Liens utiles :

- [https://mongoosejs.com/docs/api/query.html#Query.prototype.limit\(\)](https://mongoosejs.com/docs/api/query.html#Query.prototype.limit())
- [https://mongoosejs.com/docs/api/query.html#Query.prototype.select\(\)](https://mongoosejs.com/docs/api/query.html#Query.prototype.select())
- [https://mongoosejs.com/docs/api/query.html#Query.prototype.skip\(\)](https://mongoosejs.com/docs/api/query.html#Query.prototype.skip())
- [https://mongoosejs.com/docs/api/query.html#Query.prototype.sort\(\)](https://mongoosejs.com/docs/api/query.html#Query.prototype.sort())

c) Mise à jour

Avec la méthode **updateOne()** :

```
await MyModel.updateOne({ <filter> }, { <update> });
```

Met à jour le premier document qui correspond aux critères du filtre.

Avec la méthode **updateMany()** :

```
await MyModel.updateMany({ <filter> }, { <update> });
```

Met à jour tous les documents qui correspondent aux critères du filtre.

Récupérer et mettre à jour avec **findOneAndUpdate** ou **findByIdAndUpdate** :

```
// Retrouver un élément et le mettre à jour
await MyModel.findOneAndUpdate({ <filter> }, { <update> });

// Retrouver un élément par son ID et le mettre à jour
await MyModel.findByIdAndUpdate(id, { <update> });
```

Ces méthodes permettent de rechercher, de modifier et de renvoyer le document. Par défaut, elles renvoient le document dans son état avant la modification. Pour obtenir le document mis à jour, il faut ajouter l'option « new » ou « returnDocument ».

d) Remplacement

Avec la méthode **replaceOne()** :

```
await MyModel.replaceOne({ <filter> }, { <document> });
```

Remplace le premier document qui correspond au filtre par le nouveau document fourni (écrase tout le document à l'exception de l'id).

Avec la méthode **findOneAndReplace()** :

```
await MyModel.findOneAndReplace({ <filter> }, { <document> });
```

Retrouve, remplace et renvoie le document. Comme pour **findOneAndUpdate**, elle renvoie par défaut le document dans son état initial sauf si l'option « returnDocument » est spécifiée.

e) Suppression

Avec la méthode **deleteOne()** :

```
await MyModel.deleteOne({ <condition> });
```

Supprime le premier document qui correspond à la condition.

Avec la méthode [`deleteMany\(\)`](#) :

```
await MyModel.deleteMany({ <condition> });
```

Supprime tous les documents qui correspondent à la condition.

Avec la méthode [`findOneAndDelete\(\)`](#) :

```
await MyModel.findOneAndDelete({ <condition> });
```

Retrouve, supprime et renvoie le document supprimé.

Avec la méthode [`findByIdAndDelete\(\)`](#) :

```
await MyModel.findByIdAndDelete(id);
```

Retrouve, supprime et renvoie le document supprimé.

G. Middlewares

Les middlewares sont des fonctions qui sont exécutées avant (*pre*) ou après (*post*) une opération sur la base de données.

1. Différents types de middlewares

a) Les middlewares de type « Document »

Ici, le mot-clé « `this` » fait référence à l'instance du document sur laquelle le middleware est en train de s'exécuter. Ils sont supportés par les fonctions suivantes :

- [`validate`](#)
- [`save`](#)
- [`updateOne`](#), [`deleteOne`](#)
- [`init`](#) (synchrone)

b) Les middlewares de type « Modèle »

Ici, le mot-clé « `this` » fait référence au **modèle Mongoose** lui-même. Ils sont supportés par les fonctions suivantes :

- [`bulkWrite`](#)
- [`createCollection`](#)
- [`insertMany`](#)

c) Les middlewares de type « Agrégation »

Ici, le mot-clé « `this` » fait référence à l'objet d'agrégation en cours d'exécution. Ils sont supportés par la fonction [`aggregate`](#).

d) Les middlewares de type « Requête »

Ici, le mot-clé « `this` » fait référence à l'objet de requête (*Query*) sur lequel le middleware s'exécute. Ils sont supportés par les fonctions suivantes :

- [`countDocuments`](#)
- [`deleteMany`](#), [`deleteOne`](#)
- [`distinct`](#)
- [`estimatedDocumentCount`](#)
- [`find`](#), [`findOne`](#)
- [`findOneAndDelete`](#), [`findOneAndReplace`](#), [`findOneAndUpdate`](#), [`replaceOne`](#)
- [`updateOne`](#), [`updateMany`](#)
- [`validate`](#)

2. Mise en place des middlewares

Les [middlewares](#) doivent impérativement être mis en place au niveau du schéma (avant la compilation du modèle). Ils sont exécutés en série, les uns après les autres, en utilisant la syntaxe *async/await* ou des *.then/.catch*.

a) Pre

Le middleware *pre* va être utilisé pour effectuer une action **avant qu'une opération ne soit exécutée en base de données**.

Le middleware *pre* peut être utilisé par exemple pour :

- Valider ou transformer des données avant de les enregistrer.
- Hacher un mot de passe avant enregistrement.
- Faire des suppressions en cascade (ex : supprimer tous les articles d'un utilisateur lorsqu'on supprime son compte).
- Mettre à jour automatiquement la date de modification d'un document (*updatedAt*).
- Générer automatiquement des champs dérivés (comme un slug).
- Etc.

Exemple pour mettre à jour automatiquement le slug lorsque le titre change :

```
const mongoose = require('mongoose');
const slugify = require('slugify');

const articleSchema = new mongoose.Schema({
  title: String,
  content: String,
  slug: String
});

// Utilisation d'une fonction classique (pas une fonction fléchée) pour conserver le "this" du document
articleSchema.pre('save', async function() {
  if (this.isModified('title')) {
    this.slug = slugify(this.title, { lower: true });
  }
});

const Article = mongoose.model('Article', articleSchema);
```

b) Post

Le middleware *post* va être utilisé pour effectuer une action **après qu'une opération a été exécutée en base de données** (enregistrement, mise à jour, suppression).

Le middleware *post* peut être utilisé par exemple pour :

- Enregistrer des données de log dans un autre modèle ou une autre base de données.
- Envoyer une notification ou un e-mail à l'utilisateur.
- Mettre à jour des données dans un autre document ou modèle.
- Déclencher un traitement asynchrone sur des fichiers ou des images après leur enregistrement.
- Générer des données statistiques ou de suivi (analytics).
- Etc.

Exemple pour envoyer une notification après avoir créé un document :

```
articleSchema.post('save', async function(doc) {  
  sendNotification(doc.user, 'Un nouveau document a été créé.');
```

H. Requêtes avancées

a) *Populate*

[Populate](#) permet de récupérer des données à partir de plusieurs collections de la base de données via une seule requête. On l'utilise pour récupérer des données liées par une référence.

Imaginons que chaque « post » soit relié à un « author » par une référence. Pour récupérer les informations du « author » en même temps que celles du « post », on va utiliser la requête suivante :

```
await Post.find().populate('author').exec();
```

[Voir le resultat](#)

C'est ici l'équivalent d'une jointure de type « LEFT JOIN ».

b) *Agrégations*

On les utilise pour faire des calculs, des regroupements ou des traitements avancés sur les données.

L'étape de pipeline [\\$match](#) → filtrer les documents selon un critère spécifique.

```
const result = await MyModel.aggregate([  
  {  
    $match: { status: "active" } // on filtre les documents avec le statut "active"  
  },  
]);
```

L'étape de pipeline [\\$group](#) → regrouper les documents selon un champ spécifique.

```
const result = await MyModel.aggregate([  
  {  
    $group: {  
      _id: "$category",  
    }  
  }  
]);
```

Pour aller plus loin : Toutes les possibilités avec *aggregate*.

<https://mongoosejs.com/docs/api/aggregate.html>

Il est possible d'utiliser ici les [opérateurs d'agrégation](#) vus avec MongoDB.

c) *Transactions*

On utilise les [transactions](#) pour garantir l'intégrité des données lorsque plusieurs actions dépendantes sont en cours sur une base de données. Les opérations sont alors soit toutes exécutées avec succès, soit toutes annulées (principe d'atomicité).

```
try {  
  await session.withTransaction(async () => {  
    const [user] = await User.create([{ name: "John Doe" }], { session });  
    await Order.create([{ ref: "X-56G9G", user_id: user._id }], { session });  
  });  
} finally {  
  await session.endSession();  
}
```

I. Sécurité et performances

1. Les index

Les [index](#) permettent une exécution beaucoup plus efficace des requêtes. Sans index, MongoDB est obligé d'effectuer un scan complet de la collection pour sélectionner les documents demandés. Ils fonctionnent exactement comme l'index à la fin d'un livre.

Exemple de la structure avec ou sans index :

<u>_id</u>	name	email	address	status
1	Martin	martin@gmail.com	15 avenue de la République	active
2	Dupont	dupont@gmail.com	10 rue des Lilas	banned
3	Dumas	dumas@gmail.com	25 rue de la Paix	active

Figure 10: Contenu des documents. Les IDs ont été simplifiés sous forme d'entiers pour l'exemple.

status	ids
active	1, 3
banned	2

Figure 11: Index créé sur le champ status.

Imaginons que l'on ait 10 000 clients en base de données et que seulement 9 000 d'entre eux soient actifs. Si l'on effectue une requête pour cibler les utilisateurs actifs, MongoDB consultera l'index et n'ira lire que les 9 000 documents concernés. Sans index, la base de données aurait dû parcourir l'intégralité des 10 000 documents. C'est autant de lectures disques économisées.

Il y a cependant des contreparties importantes à connaître :

- Espace de stockage : plus on crée d'index, plus le poids de la base de données augmente.
- Écritures ralenties : l'index doit être mis à jour en temps réel dès qu'un document est créé, modifié ou supprimé. Cela ralentit légèrement les opérations d'écriture.

Tout est une question de compromis entre la vitesse de lecture et la vitesse d'écriture !

Ajout sur le schéma :

[https://mongoosejs.com/docs/api/schematype.html#SchemaType.prototype.index\(\)](https://mongoosejs.com/docs/api/schematype.html#SchemaType.prototype.index())

Pour aller plus loin : Les index « sparse »

Les [index « sparse »](#) ne recensent que les documents qui contiennent explicitement le champ indexé.

Dans l'exemple ci-dessus, si certains clients n'avaient aucun champ « status » défini, l'index ignorerait ces documents. Cela permet d'alléger considérablement la taille de l'index en mémoire lorsque le champ est optionnel ou rarement renseigné dans la collection.

Ajout sur le schéma :

[https://mongoosejs.com/docs/api/schematype.html#SchemaType.prototype.sparse\(\)](https://mongoosejs.com/docs/api/schematype.html#SchemaType.prototype.sparse())

2. Sécurité

- **Injections NoSQL** : bien que Mongoose utilise des objets JavaScript et élimine de fait les risques d'injections SQL traditionnelles, il reste sensible aux injections NoSQL. Un attaquant peut injecter des opérateurs MongoDB via des requêtes mal nettoyées pour contourner une authentification.
→ Pour s'en prémunir, il est recommandé de valider strictement le type des données entrantes, d'utiliser des middlewares comme [express-mongo-sanitize](#) pour filtrer les clés suspectes, et de ne jamais passer directement *req.body* ou *req.query* à une méthode de requête sans les avoir préalablement contrôlés.
- **Protection des comptes et données** : face aux attaques par force brute sur l'API.
→ Il convient de mettre en place une limitation du taux de requêtes ([rate-limiting](#)). En base de données, les mots de passe doivent impérativement être [hachés](#) et un validateur doit être configuré au niveau du schéma pour exiger des mots de passe complexes.
- **Attaques de type Man-in-the-Middle** : interception des données en transit.
→ L'utilisation d'une [connexion chiffrée SSL/TLS](#) est indispensable entre l'API Node.js et le serveur de base de données MongoDB afin de sécuriser le transit des données.
- **Limitation des privilèges (Moindre privilège)** : restriction des accès au niveau du serveur MongoDB.
→ L'API ne doit jamais se connecter avec un accès administrateur (*root* global). Elle doit utiliser un [compte utilisateur dédié](#) à l'application, doté des seuls droits nécessaires à son fonctionnement (*readWrite* sur sa propre base de données).

La sécurité de la base de données est indissociable de celle de l'application. L'API doit elle-même intégrer ses propres mécanismes de défense (pare-feu, configuration stricte du CORS, protection contre les failles CSRF, etc.).

En savoir plus : <https://www.mongodb.com/docs/manual/administration/security-checklist/>

3. Performance

- **Utilisation des index** : les index réduisent drastiquement le temps de recherche des informations et améliorent ainsi les performances des requêtes en lecture.
- **Mise en place de la pagination** : il convient de limiter la quantité de données récupérées simultanément par une seule requête pour éviter de surcharger la mémoire de l'API et de la base de données.
- **Utilisation d'un système de cache** : la mise en mémoire tampon (via un outil comme Redis) des données fréquemment lues et qui varient peu permet de réduire le nombre de requêtes directes vers MongoDB.
- **Éviction des requêtes coûteuses** : il est recommandé d'éviter les requêtes impliquant trop de niveaux d'imbrication, de jointures répétées ou d'agrégations complexes sur de grands volumes de données.
- **Optimisation des schémas** : la conception du schéma doit faire un choix stratégique entre documents imbriqués et références. L'objectif est de réduire la taille globale des documents, d'utiliser les types de données appropriés et de restreindre le nombre de champs indexés aux stricts besoins de l'application.

Pour aller plus loin : Accélérer les requêtes avec *lean()*

Par défaut, Mongoose convertit chaque document retourné en une instance de document Mongoose (qui inclut le suivi des modifications, les getters/setters, les méthodes de sauvegarde, etc.). Cela consomme beaucoup de mémoire et de temps CPU.

L'utilisation de la fonction [*.lean\(\)*](#) permet de retourner les résultats d'une requête sous forme d'objets JavaScript bruts. En évitant l'instanciation complète du modèle Mongoose, les performances de lecture peuvent être multipliées par 4 ou 5.